

23CSE203 DATA STRUCTURES AND ALGORITHM

Submitted by	
Name	J.Rudrateja
Roll.no	AV.SC.U4CSE25114
Section	CSE-B

SUBMITTED TO	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR (Sl. Grade)
DEPARTMENT	SCHOOL OF COMPUTING

Credit	5

Index

S.no	Title	Page.no	Remark
1.	A) A rocket launch system displays a countdown before launch. Instead of using loop, implement the countdown using recursion. Write a recursive program to display a countdown from n to 1 followed by launch. B) Bank calculates the growth of an investment using compound intrest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n(power), where P=principal growth factor, n=no.of years. C) Write a recursive progrsm to print the first n terms of fibanocci series **D)** An hr department stores employee ids in a list develop a recursive program to search for a given employee id. Write a recursive function to search for an employee id in the list. E) In a logistics the no.of possible ways to arrange n different parcels calculate using the factorial (n!). develop a recursive program to calculate the factorial of given number.	4-13	
2.	A) write a python program to perform linear search on the given input B) Write a python program to perform binary search on the given	14-18	

	input. Verify by taking sorted array and unsorted array as a list.		
3.	A) write a python program to implement bubble sort, where the elements are taken from user. B) write a python program to implement selection sort, where the elements are taken from user. C) write a python program to implement insertion sort, where the elements are taken from user.	19-	
5.	Aim 5.a:Write a program to implement a single linked list which would perform the below operations, by taking an input from the user. a) create a linked list b) insert at beginning c) insert at end d) insert at a specified index e) delete by value f) delete 1st node g) delete last node h) count no.of nodes i) display or traverse j) search for a number k)exit		
4.	A) Write a python program to sort the elements using Quick sort technique. Take an input from the user. B) Write a python program to sort the elements using Merge sort technique. Take an input from the user.		
6.	A) Write a python programme to implement queue and its operations such as nQand dQ peak and and display with menu driven approach implement menu using arrays linked list data structures . B) Write a python programme to implement Circular queue and its operations such as nQand dQ peak and and display with menu driven approach implement menu using arrays linked list data structures .		

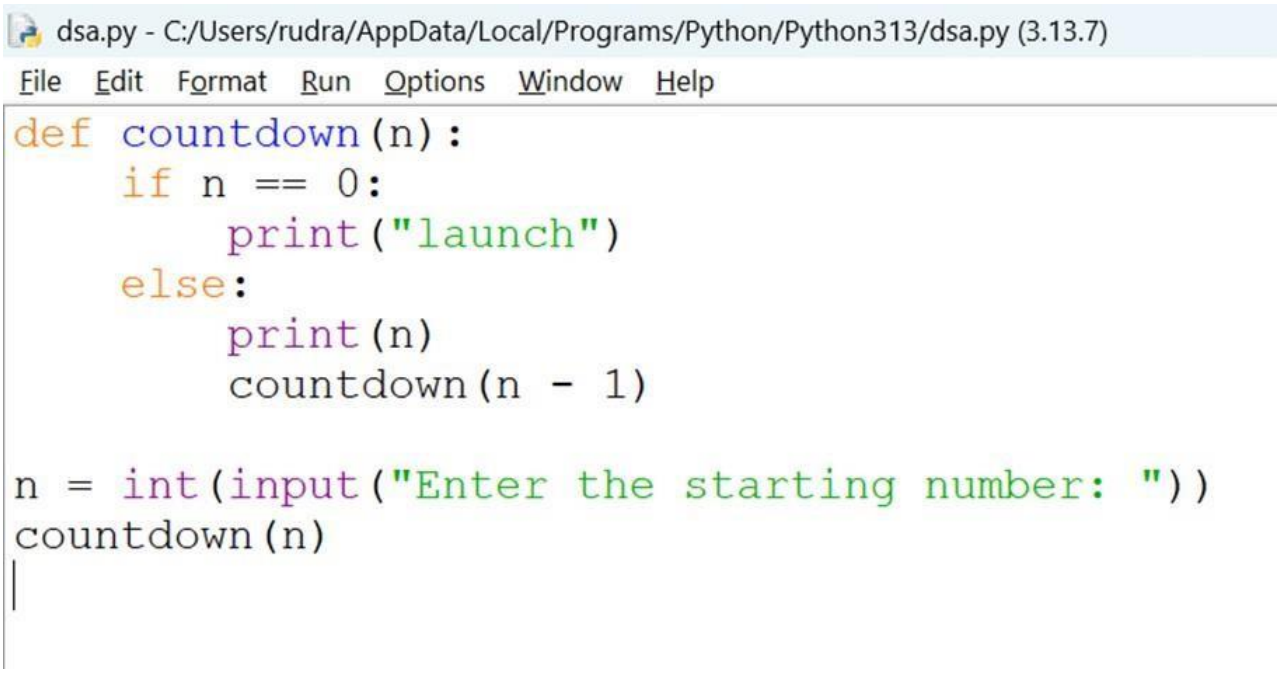
	Programme:		
7.	a. Write a Python program to implement a Queue and its operations such as Enqueue, Dequeue, Peek, and Display using a menu-driven approach. Implement the Queue using both an Array and a Linked List data structure. b. Write a Python program to implement a Circular Queue and its operations such as Enqueue, Dequeue, Peek, and Display using a menu-driven approach.		
8.	Write a Python program to implement a Binary Tree with its operations using an Array and a Linked List.		
9.	write a python programme to implement a binary search tree with its operations and traversals.		

Week-1

A) LAUNCH:

Aim: A rocket launch system displays a countdown before launch. Instead of using loop, implement the countdown using recursion. Write a recursive program to display a countdown from n to 1 followed by launch.

Program:



Output:

```
==== RESTART: C:/Users/rudra/A
Enter the starting number: 24
24
23
22
21
20
19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
launch
```

Explanation:

- It takes a number as input from the user and stores it in num.
- It creates a function named launch().
- It starts the countdown from 10 by assigning i = 10.
- It prints the value of i as long as num <= i.
- It decreases the value of i by 1 after each iteration.
- When the loop ends, it prints "Rocket launched" and executes the function using launch().
- For negative input, the function keeps decreasing instead of reaching 0, causing infinite recursion and eventually a Recursion Error.

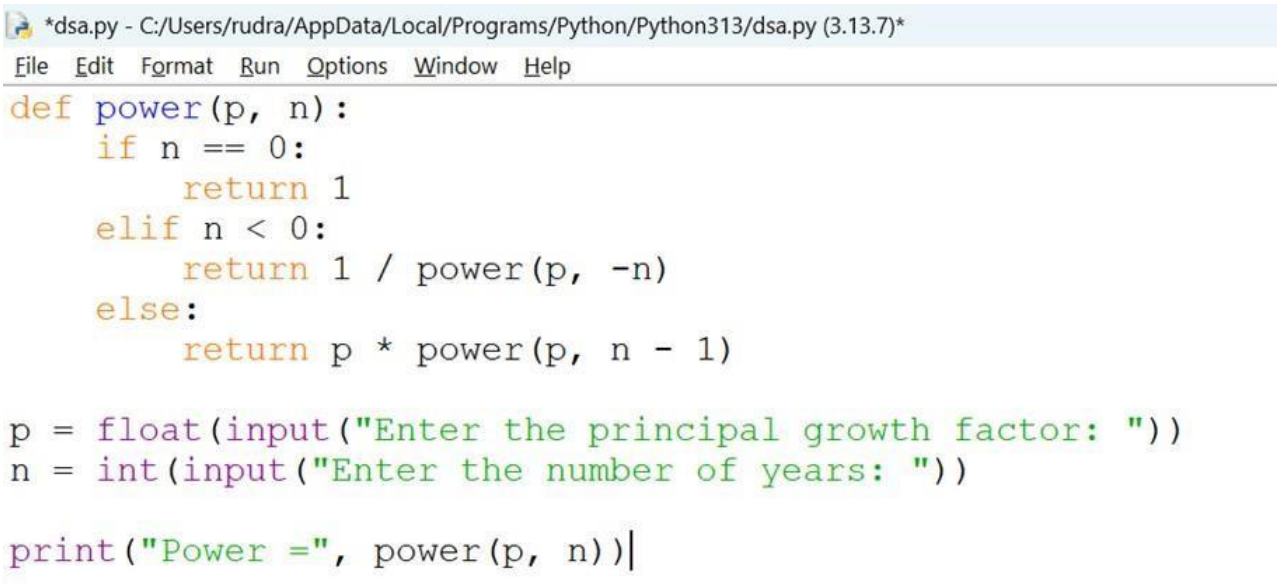
Error table:

s.no	Error obtained	Rectification
1.	Infinite loop	Iteration at correct place
2.	Calling function at wrong place	Placed with correct syntax

B) POWER:

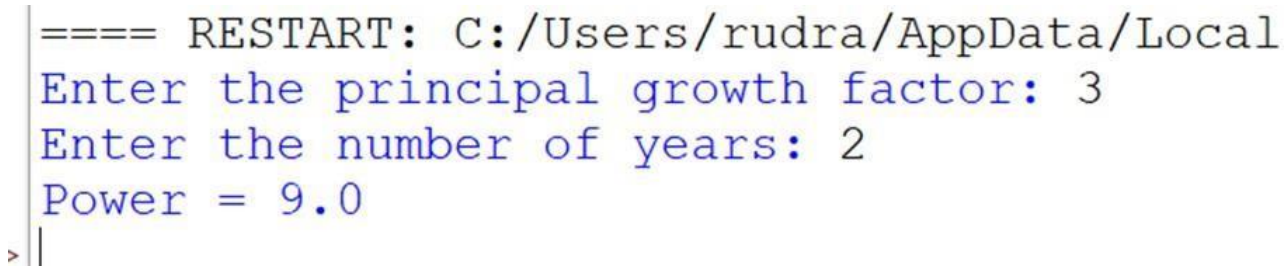
Aim: Bank calculates the growth of an investment using compound intrest. The formula requires Calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n(power), where P=principal growth factor, n=no.of years.

Program:



OUTPUT:

Output:



Explanation:

- The program takes the principal growth factor p and number of years n as input.
- It defines a recursive function power(p, n). • If n is 0, the function returns 1 (base case).
- Otherwise, it returns p * power(p, n-1).
- The recursive calls continue until n becomes 0.
- The final value of p^n is displayed as the output.
- For a negative exponent, calculate the reciprocal instead of continuing recursion downward indefinitely.
- Error table:

s.no	Error obtained	Rectification
1.	Return 0	Return 1
2.	print	return

C)FIBANOCCI

Aim: Write a recursive program to print the first n terms of fibanocci series Program:

PROGRAMME:

dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)

File Edit Format Run Options Window Help

```
def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input("Enter number of terms: "))

for i in range(n):
    print(fibonacci(i), end=" ")
```

OUTPUT:

```
==== RESTART: C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py ==
Enter number of terms: 10
0 1 1 2 3 5 8 13 21 34
```

Explanation:

- For entering the number as input from user we defined as n
- We declared variable for formula.
- If input number is less than one then it return “1” • Or it will goes under fibanocci iteration.
- If x is not present in the list, recursion reaches the end and returns False. Error:

s.no	Error obtained	Rectification
1.	Forgot to declare n<=1	rectified
2.	Formula mistake	rectified

D)EMPLOYEE ID:

Aim: An hr department stores employee ids in a list develop a recursive program to search for a given employee id. Write a recursive function to search for an employee id in the list.

Program:

dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)

File Edit Format Run Options Window Help

```
def search(emp_list, x, i=0):
    if i == len(emp_list):
        return False

    if emp_list[i] == x:
        return True

    return search(emp_list, x, i + 1)


emp_list = [101, 205, 310, 415, 520]

x = int(input("Enter employee ID to search: "))

if search(emp_list, x):
    print("Employee ID found.")
else:
    print("Employee ID not found.")
```

Output:

```
=== RESTART: C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py =
Enter employee ID to search: 101
Employee ID found.
```

Explanation:

- The program creates a function to search for an employee ID in a list.
- It checks one employee ID at a time, starting from the first ID.
- If the entered employee ID matches the current ID, it returns True.
- If it does not match, the function moves to the next ID using recursion.
- If all employee IDs are checked and no match is found, it returns False.
- Finally, the program displays "Employee ID found." if the ID exists; otherwise, it displays "Employee ID not found."
- ERROR TABLE:

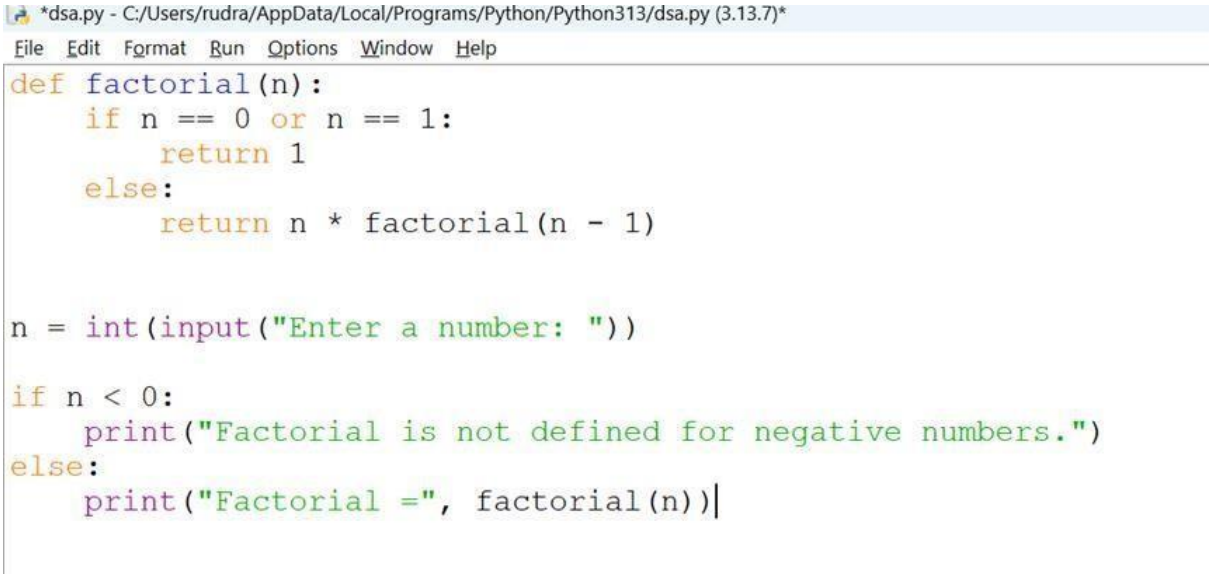
s.no	Error obtained	Rectification
1.	Emp lst	Emp_list
2.	print	Return

E)Recursive Factorial :

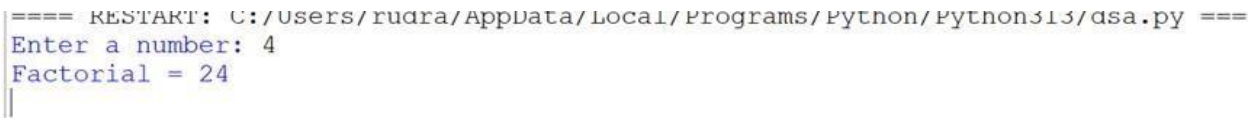
Aim: In a logistics the no.of possible ways to arrange n different parcels calculate using the factorial (n!)

. develop a recursive program to calculate the factorial of given number.

Program:



Output:



Explanation:

- The program defines a recursive function fact() to calculate the factorial of a number.
- It takes a number as input from the user.
- If the number is 1, the function returns 1 (base case).
- Otherwise, it multiplies the number by the factorial of the previous number (n-1).

- The recursive calls continue until the number becomes 1.
- Finally, the program prints the factorial of the given number.
- Negative numbers do not have a factorial, so the program should handle them separately instead of recursing forever.

Error table:

s.no	Error obtained	Rectification
1.	n+1	n-1
2.	Syntax error	Rectified

WEEK-2

A)LINEARSEARCH

AIM: Write a python program to perform linear search on the given input.

Program:

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1


arr = list(map(int, input("Enter elements separated by space: ").split()))
key = int(input("Enter the element to search: "))

result = linear_search(arr, key)

if result == -1:
    print("Element not found")
else:
    print("Element found at index:", result)
```

OUTPUT:

Output:

```
==== RESTART: C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py =
Enter elements separated by space: 3 3
Enter the element to search: 3
Element found at index: 0
>
```

Explanation:

- In linear search we can append elements in sorted or unsorted way also.
- After placing the target we need to check the all the elements in a order.
- The time complixity is O(n).
- The target element may be found at beginning or last or at middle.
- If the key is greater than every element in the sorted array, the search reaches the end and returns -1.

Error table:

s.no	Error obtained	Rectification
1.	Return placed in wrong place	Corrected later
2.	Calling the function	Rectified

B)BINARY SEARCH

Aim: Write a python program to perform binary search on the list.

Program: Unsorted

dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)

File Edit Format Run Options Window Help

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1


arr = list(map(int, input("Enter elements separated by space: ").split()))

arr.sort()

print("Sorted array:", arr)

key = int(input("Enter the element to search: "))

result = binary_search(arr, key)

if result == -1:
    print("Element not found")
else:
    print("Element found at index:", result)
```

OUTPUT:

Output:

```
==== RESTART: C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py ====
Enter elements separated by space: 12
Sorted array: [12]
Enter the element to search: 1
Element not found
```

PROGRAM: SORTED

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1


arr = list(map(int, input("Enter sorted elements separated by space: ").split()))

key = int(input("Enter the element to search: "))

result = binary_search(arr, key)

if result == -1:
    print("Element not found")
else:
    print("Element found at index:", result)
```

OUTPUT:

Output:

```
==== RESTART: C:/Users/rudra/AppData/Local/Programs/Py
Enter sorted elements separated by space: 1 2 3 4 5 6
Enter the element to search: 4
Element found at index: 3
```

Explanation:

- The binary search should always executed in sorted order.
- Either it can be ascending or descending order.
- So the program is in unsorted order so we need to do sorting first later the binary search is executed.
- Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

WEEK-3

A)Bobble sort:

AIM: write a python program to implement bubble sort, where the elements are taken from user.

Program:

dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)

File Edit Format Run Options Window Help

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]


n = int(input("Enter no. of elements: "))

arr = []

print("Enter the elements:")

for i in range(n):
    arr.append(int(input()))

bubble_sort(arr)

print("Sorted Array:")

for element in arr:
    print(element, end=" ")
```

Output:

```
==== RESTART: C:/Users/rudra/
Enter no. of elements: 4
Enter the elements:
1
2
6
8
Sorted Array:
1 2 6 8
```

Explanation:

- Bubble sort is a sorting technique used to arrange elements in **ascending or descending order**.
- The program may contain elements in **unsorted order**, so adjacent elements are compared and swapped if they are in the wrong order.
- This process is repeated for multiple passes until the **entire array is sorted**.
- For n = 0, the program now prints the error message without trying to use an undefined arr.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

B)Insertion sort:

AIM: write a python program to implement insertion sort, where the elements are taken from user.

Program:

dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)

File Edit Format Run Options Window Help

```
def insertion_sort(arr):
    n = len(arr)

    for i in range(1, n):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j = j - 1

        arr[j + 1] = key


n = int(input("Enter no. of elements: "))

arr = []

print("Enter the elements:")

for i in range(n):
    arr.append(int(input()))

insertion_sort(arr)

print("Sorted Array:")

for element in arr:
    print(element, end=" ")
```

OUTPUT:

```
==== RESTART: C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py
Enter no. of elements: 2
Enter the elements:
1
2
Sorted Array:
1 2
```

Explanation:

- Insertion sort is a sorting technique used to arrange elements in **ascending or descending order**.
- The program takes one element at a time and **inserts it into its correct position** in the already sorted part.
- This process is repeated until **all elements are sorted**.
- When the number of elements is zero, the array is empty and insertion sort simply performs no iterations.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

C)Selection sort:

AIM: write a python program to implement selection sort, where the elements are taken from user.

Program:

dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)

File Edit Format Run Options Window Help

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]


n = int(input("Enter no. of elements: "))

arr = []

print("Enter the elements:")

for i in range(n):
    arr.append(int(input()))

selection_sort(arr)

print("Sorted Array:")

for element in arr:
    print(element, end=" ")
```

Output:

```
==== RESTART: C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py ===
Enter no. of elements: 3
Enter the elements:
5
4
3
Sorted Array:
3 4 5
```

EXPLANATION:

- Selection sort is a sorting technique used to arrange elements in **ascending or descending order**.
- The program finds the **smallest element** from the unsorted part and swaps it with the first element of that part.
- This process is repeated until the **entire array is sorted**.
- When n = 0, the array is empty and selection sort performs no iterations, so the program runs normally.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

WEEK 4

AIM 1.a) Write a python program to sort the elements using Quick sort technique. Take an input from the user.

PROGRAM:

```
print("the new sorted array is",result)
```

Output:

```
enter no.of elements to add5
enter a element:12
enter a element:-2
enter a element:45
enter a element:14
enter a element:18
the original array is [12, -2, 45, 14, 18]
the new sorted array is [-2, 12, 14, 18, 45]
```

Explanation:

This program sorts the elements using Quick Sort. It selects a pivot element and partitions the list so that smaller elements are placed on one side and larger elements on the other side. The same process is then applied recursively to the two parts.

Error Table:

ERROR	CAUSE	SOLUTION
Recursion Error	Recursive calls do not reach the base condition.	Ensure the partition reduces the problem size.
Index Error	Incorrect low, high, or partition boundaries.	Check the partition indices and recursive ranges.
Logic Error	Pivot partitioning is performed incorrectly.	Check the pivot comparison and partition logic.

AIM 1.b) Write a python program to sort the elements using Merge sort technique. Take an input from the user.

Program:

```
print("the new sorted array is",result)
```

Output:

```
enter no.of elements to add4
enter a element:-9
enter a element:58
enter a element:14
enter a element:23
the original array is [-9, 58, 14, 23]
the new sorted array is [-9, 14, 23, 58]
```

Explanation:

This program sorts the elements using Merge Sort. It divides the list into smaller halves until single elements remain, then merges the smaller sorted parts back together in the correct order to obtain the final sorted list.

Error Table:

ERROR	CAUSE	SOLUTION
Index Error	Incorrect indices are used while splitting or merging.	Check the sub-list boundaries and indices.
Value Error	Invalid non-integer input is entered.	Enter valid integer values.
Logic Error	The two sorted parts are not merged correctly.	Check the comparison and merging steps.

TIME COMPLEXITY AND SPACE COMPLEXITY

PROGRAM	BEST TIME	AVERAGE TIME	WORST TIME	SPACE COMPLEXITY	REASON

1. Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Adjacent elements are compared repeatedly. With an optimized version, an already sorted list can finish

					in one pass.
2. Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Each element is inserted into its correct position. It is fast when the input is already or nearly sorted.
3. Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	The minimum element is searched in the remaining unsorted part during every pass.
4. Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	The array is partitioned around a pivot. Balanced partitions give O(n log n), while highly unbalanced partitions give O(n^2).
5. Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	The list is divided into halves and the sorted halves

WEEk -5

Aim 5a:Write a program to implement a single linked list which would perform the below operations, by taking an input from the user.

a) **create a linked list**
b) **insert at beginning**
c) **insert at end**
d) **insert at a specified index**
e) **delete by value**
f) **delete 1st node**
g) **delete last node**
h) **count no.of nodes**
i) **display or traverse**
j) **search for a number**
k) **exit**

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SLL:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head

                while temp.next is not None:
                    temp = temp.next

                temp.next = new_node

        print("Linked List created successfully")

    # 2. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))

        new_node = Node(data)

        new_node.next = self.head
        self.head = new_node

        print(f"Inserted {data} at the beginning.")

    # 3. Insert at End
    def insert_end(self):
        data = int(input("Enter data: "))

        new_node = Node(data)
```

File Edit Format Run Options Window Help

```
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            print(f"Inserted {data} at the end.")
            return

        temp = self.head

        while temp.next is not None:
            temp = temp.next

        temp.next = new_node

        print(f"Inserted {data} at the end.")

    # 4. Insert at Index
    def insert_at_index(self):
        data = int(input("Enter data: "))
        index = int(input("Enter index: "))

        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            new_node = Node(data)
            new_node.next = self.head
            self.head = new_node

            print(f"Inserted {data} at index {index}.")
            return

        temp = self.head

        for i in range(index - 1):
            if temp is None:
                print("Index out of range")
                return

            temp = temp.next

        if temp is None:
            print("Index out of range")
            return

        new_node = Node(data)
```

```
        while temp is not None:

            if temp.data == data:
                print("Element found at index", index)
                return

            temp = temp.next
            index += 1

        print("Element not found")

# Main Program
```

```
sll = SLL()

while True:

    print("\n----- SINGLY LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete from Beginning")
    print("6. Delete from End")
    print("7. Delete from Index")
    print("8. Display")
    print("9. Count Nodes")
    print("10. Search")
    print("11. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        sll.create()

    elif choice == 2:
        sll.insert_beginning()

    elif choice == 3:
        sll.insert_end()

    elif choice == 4:
        sll.insert_at_index()

    elif choice == 5:
        sll.delete_beginning()

    elif choice == 6:
        sll.delete_end()

    elif choice == 7:
        sll.delete_at_index()

    elif choice == 8:
        sll.display()

    elif choice == 9:
        sll.count_nodes()
```

```
    elif choice == 9:
        sll.count_nodes()

    elif choice == 10:
        sll.search()

    elif choice == 11:
        print("Exiting from the program...")
        break

    else:
        print("Invalid choice")
```

Output:

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Display
9. Count Nodes
10. Search
11. Exit
Enter your choice: 3
Enter data: 11
Inserted 11 at the end.

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Display
9. Count Nodes
10. Search
11. Exit
Enter your choice: 4
Enter data: 56
Enter index: 1
Inserted 56 at index 1.

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Display
9. Count Nodes
10. Search
11. Exit
Enter your choice: 5
Deleted 7 from beginning.

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Display
9. Count Nodes
10. Search
11. Exit
Enter your choice: 9
Number of nodes: 5

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Display
9. Count Nodes
10. Search
11. Exit
Enter your choice: 6
Deleted 11 from end.

----- SINGLY LINKED LIST -----
```

```
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Display
9. Count Nodes
10. Search
11. Exit
Enter your choice: 7
Enter index: 3
Deleted 4 from index 3.

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Display
9. Count Nodes
10. Search
11. Exit
Enter your choice: 8
56 -> 12 -> 3 -> None

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Display
9. Count Nodes
6. Delete from End
7. Delete from Index
8. Display
9. Count Nodes
10. Search
11. Exit
Enter your choice: 9
Number of nodes: 3

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Display
9. Count Nodes
10. Search
11. Exit
Enter your choice: 10
Enter element to search: 112
Element not found

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Display
9. Count Nodes
10. Search
11. Exit
Enter your choice: 11
Exiting from the program...
```

Explanation:

A **Singly Linked List (SLL)** is a linear data structure in which each node contains two parts: **data** and a **next pointer**. The data part stores the value, while the next part stores the address/reference of the next node. The head pointer points to the first node of the linked list. The last node points to None, indicating the end of the list.

The program implements the following operations:

- **Create Linked List:** Creates a linked list by taking the number of nodes and their data from the user. Each new node is connected to the previous node.
- **Insert at Beginning:** Creates a new node and makes it the first node by changing the head pointer.
- **Insert at End:** Traverses the list until the last node and connects the new node to it.
- **Insert at Index:** Inserts a new node at a specified position by adjusting the links between nodes.
- **Delete from Beginning:** Removes the first node by moving the head pointer to the next node.
- **Delete from End:** Traverses to the second-last node and changes its next pointer to None.
- **Delete from Index:** Removes a node from a specified position by changing the next pointer of the previous node.
- **Display:** Traverses the complete linked list and displays each node's data.
- **Search:** Traverses the list and checks each node to find the required element. It also displays its index if found.
- **Count Nodes:** Traverses the list using a temporary pointer and increments count for every node until None is reached.
- **Exit:** Terminates the program.
- The program uses a **menu-driven approach**, allowing the user to select and perform different linked-list operations repeatedly.

ERROR TABLE :

PROGRAM	ERROR / PROBLEM	CAUSE	CORRECTION
1. Create Linked List	List is not created properly	head or next links are not initialized correctly	Initialize head = None and connect each new node using next.
2. Insert at Beginning	New node is not inserted at the beginning	head is not updated correctly	Set new_node.next = self.head and then self.head = new_node.
3. Insert at End	New node is not added at the end	Traversal or next assignment is incorrect	Traverse until temp.next is None and set temp.next = new_node.
4. Insert at Index	Node is inserted at the wrong position	Incorrect index traversal	Traverse up to index - 1 and adjust the next

			links.
5. Delete from Beginning	First node is not deleted	head is not moved to the next node	Set self.head = self.head.next.
6. Delete from End	Last node is not deleted	Second-last node is not identified correctly	Traverse until temp.next.next is None and set temp.next = None.
7. Delete from Index	Wrong node is deleted	Incorrect index or link modification	Traverse to index - 1 and update temp.next = temp.next.next.
8. Display	List elements are not displayed completely	temp is not updated during traversal	Use temp = temp.next until temp is None.
9. Search	Element is not found correctly	List is not traversed completely	Compare each node and update temp = temp.next.
10. Count Nodes	Incorrect number of nodes is displayed	Last node is skipped during counting	Use while temp is not None and increment count for every node.

TIME COMPLEXITY AND SPACE COMPLEXITY

PROGRAM	**BEST TIME**	**AVERAGE TIME**	**WORST TIME**	**SPACE COMPLEXITY**	**REASON**

1. Create Linked List	O(n)	$O(n^2)$	$O(n^2)$	O(n)	Each node is created and connected. Traversing to the end for every insertion makes creation $O(n^2)$.
2. Insert at Beginning	O(1)	O(1)	O(1)	O(1)	A new node is directly connected to the current head.
3. Insert at End	O(n)	O(n)	O(n)	O(1)	The list is traversed until the last node before insertion.
4. Insert at Index	O(1)	O(n)	O(n)	O(1)	The list may need to be traversed up to the required index.
5. Delete from Beginning	O(1)	O(1)	O(1)	O(1)	Only the head pointer is moved to the next node.
6. Delete from End	O(n)	O(n)	O(n)	O(1)	The list must be traversed to find the second-last node.

7. Delete from Index	O(1)	O(n)	O(n)	O(1)	Traversal may be required to reach the node before the specified index.
8. Display	O(n)	O(n)	O(n)	O(1)	Every node is visited once to display its data.
9. Search	O(1)	O(n)	O(n)	O(1)	The element may be
10. Count Nodes	O(n)	O(n)	O(n)	O(1)	Every node is visited once to calculate the total count.

WEEK 6:

AIM: 6A) Write a python programme to implement queue and its operations such as nQand dQ peak and display with menu driven approach implement menu using arrays linked list data structures .

Programme:

```
            self.rear = None
```

```
            print(f"Dequeued (LL): {dequeued_item}")
            return dequeued_item

        def peak(self):
            if self.is_empty():
                print("Queue is empty.")
                return None
            return self.front.data

        def display(self):
            if self.is_empty():
                print("Queue is empty.")
                return
            temp = self.front
            print("Queue (Linked List) [Front -> Rear]: ", end="")
            while temp:
                print(temp.data, end=" <- ")
                temp = temp.next
            print("None")

# Queue using Array (Python List)
class QueueArray:
    def __init__(self):
        self.queue = []

    def is_empty(self):
        return len(self.queue) == 0

    def nQ(self, item):
        self.queue.append(item)
        print(f"Enqueued (Array): {item}")
```

```
    def dQ(self):
        if self.is_empty():
            print("Queue Underflow! Queue is empty.")
            return None
        dequeued_item = self.queue.pop(0)
        print(f"Dequeued (Array): {dequeued_item}")
        return dequeued_item

    def peak(self):
        if self.is_empty():
            print("Queue is empty.")
            return None
        return self.queue[0]

    def display(self):
        if self.is_empty():
            print("Queue is empty.")
            return
        print("Queue (Array) [Front -> Rear]:", self.queue)

# Menu-driven runner function
def main():
    print("Select Queue Implementation:")
    print("1. Array (Python List)")
    print("2. Linked List")
    choice = input("Enter choice (1 or 2): ").strip()

    if choice == '1':
        q = QueueArray()
        name = "Array"
    elif choice == '2':
```

```
elif choice == '2':
    q = QueueLinkedList()
    name = "Linked List"
else:
    print("Invalid choice. Exiting.")
    return

while True:
    print(f"\n--- Queue Menu ({name} Implementation) ---")
    print("1. nQ (Enqueue)")
    print("2. dQ (Dequeue)")
    print("3. Peak")
    print("4. Display")
    print("5. Exit")

    opt = input("Enter your option (1-5): ").strip()

    if opt == '1':
        val = input("Enter value to enqueue: ")
        q.nQ(val)
    elif opt == '2':
        q.dQ()
    elif opt == '3':
        front_val = q.peak()
        if front_val is not None:
            print(f"Peak element (Front): {front_val}")
    elif opt == '4':
        q.display()
    elif opt == '5':
        print("Exiting program.")
        break
    else:
        print("Invalid option! Please choose between 1 and 5.")
```


```
    elif opt == '5':
        print("Exiting program.")
        break
    else:
        print("Invalid option! Please choose between 1 and 5.")

__name__ == "__main__":
main()
```

Output:

```
Select Queue Implementation:
1. Array (Python List)
2. Linked List
Enter choice (1 or 2): 1

--- Queue Menu (Array Implementation) ---
1. nQ (Enqueue)
2. dQ (Dequeue)
3. Peak
4. Display
5. Exit
Enter your option (1-5): 2
Queue Underflow! Queue is empty.

--- Queue Menu (Array Implementation) ---
1. nQ (Enqueue)
2. dQ (Dequeue)
3. Peak
4. Display
5. Exit
Enter your option (1-5): 3
Queue is empty.

--- Queue Menu (Array Implementation) ---
1. nQ (Enqueue)
2. dQ (Dequeue)
3. Peak
4. Display
5. Exit
Enter your option (1-5): 4
Queue is empty.
--- Queue Menu (Array Implementation) ---
1. nQ (Enqueue)
2. dQ (Dequeue)
3. Peak
4. Display
5. Exit
Enter your option (1-5): 4
Queue is empty.

--- Queue Menu (Array Implementation) ---
1. nQ (Enqueue)
2. dQ (Dequeue)
3. Peak
4. Display
5. Exit
Enter your option (1-5): 5
Exiting program.
```

Explanation:

Circular Queue using Linked List

In a linear linked list queue, the rear node points to None. In a Circular Queue using a Linked List, the rear node's next pointer loops back to point directly to the front node. This creates a continuous cycle.

- **Empty State: Both front and rear pointers point to None.**

- **Enqueue (nQ): A new node is allocated. If the queue is empty, both pointers target this node, and its next pointer targets itself. If nodes already exist, the current rear.next links to the new node, rear moves to this new node, and the new rear.next points back to front.**
- **Dequeue (dQ): The element at front is safely removed. If it was the last node (front == rear), both pointers reset back to None. Otherwise, front shifts to front.next, and rear.next is updated to point to the newly assigned front node to preserve structural circularity.**

Logic & Operational Error Table

This table maps common conceptual mistakes, implementation bugs, and runtime risks when working with a Circular Linked List Queue, along with their solutions.

AIM 6B) Write a python programme to implement Circular queue and its operations such as nQand dQ peak and and display with menu driven approach implement menu using arrays linked list data structures

.

Programme:

```
# Node class for Circular Queue Linked List implementation
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# 1. Circular Queue using Linked List
class CircularQueueLinkedList:
    def __init__(self):
        self.front = None
        self.rear = None

    def is_empty(self):
        return self.front is None

    def nQ(self, item):
        new_node = Node(item)
        if self.is_empty():
            self.front = new_node
            self.rear = new_node
            self.rear.next = self.front  # Point to itself to make it circular
        else:
            self.rear.next = new_node
            self.rear = new_node
            self.rear.next = self.front  # Maintain circular link
        print(f"Enqueued (LL): {item}")

    def dQ(self):
        if self.is_empty():
            print("Queue Underflow! Circular Queue is empty.")
            return None
```

```
        dequeued_item = self.front.data

        # If there is only one element in the queue
        if self.front == self.rear:
            self.front = None
            self.rear = None
        else:
            self.front = self.front.next
            self.rear.next = self.front  # Maintain circular link

        print(f"Dequeued (LL): {dequeued_item}")
        return dequeued_item

    def peak(self):
        if self.is_empty():
            print("Queue is empty.")
            return None
        return self.front.data

    def display(self):
        if self.is_empty():
            print("Queue is empty.")
            return

        print("Circular Queue (Linked List) [Front -> Rear]: ", end="")
        temp = self.front
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.front:
                break
        print("(Back to Front: " + str(self.front.data) + ")")
```

```
55
56
57 # 2. Circular Queue using Array (Fixed-size list)
58 class CircularQueueArray:
59     def __init__(self, size):
70         self.size = size
71         self.queue = [None] * size
72         self.front = -1
73         self.rear = -1
74
75     def is_full(self):
76         return (self.rear + 1) % self.size == self.front
77
78     def is_empty(self):
79         return self.front == -1
80
81     def nQ(self, item):
82         if self.is_full():
83             print("Queue Overflow! Circular Queue is full.")
84             return
85
86         if self.front == -1:  # First element insertion
87             self.front = 0
88
89         self.rear = (self.rear + 1) % self.size
90         self.queue[self.rear] = item
91         print(f"Enqueued (Array): {item}")
92
93     def dQ(self):
94         if self.is_empty():
95             print("Queue Underflow! Circular Queue is empty.")
96             return None
```

```
    def dQ(self):
        if self.is_empty():
            print("Queue Underflow! Circular Queue is empty.")
            return None

        dequeued_item = self.queue[self.front]
        self.queue[self.front] = None  # Clear memory slot visually

        if self.front == self.rear:  # Element was the last remaining item
            self.front = -1
            self.rear = -1
        else:
            self.front = (self.front + 1) % self.size

        print(f"Dequeued (Array): {dequeued_item}")
        return dequeued_item

    def peak(self):
        if self.is_empty():
            print("Queue is empty.")
            return None
        return self.queue[self.front]

    def display(self):
        if self.is_empty():
            print("Queue is empty.")
            return

        print("Raw Array State:", self.queue)
        print("Circular Queue (Array elements from Front to Rear): ", end="")
```

```
            return

        print("Raw Array State:", self.queue)
        print("Circular Queue (Array elements from Front to Rear): ", end="")

        temp = self.front
        while True:
            print(self.queue[temp], end=" ")
            if temp == self.rear:
                break
            temp = (temp + 1) % self.size
        print()


# Menu-driven runner function
def main():
    print("Select Circular Queue Implementation:")
    print("1. Array (Fixed-size Python List)")
    print("2. Linked List")
    choice = input("Enter choice (1 or 2): ").strip()

    if choice == '1':
        try:
            size = int(input("Enter the fixed capacity of the circular array: "))
            if size <= 0:
                raise ValueError
        except ValueError:
            print("Invalid size! Defaulting size to 5.")
            size = 5
        q = CircularQueueArray(size)
        name = "Array"
    elif choice == '2':
```

```
        q = CircularQueueLinkedList()
        name = "Linked List"
    else:
        print("Invalid choice. Exiting.")
        return

    while True:
        print(f"\n--- Circular Queue Menu ({name} Implementation) ---")
        print("1. nQ (Enqueue)")
        print("2. dQ (Dequeue)")
        print("3. Peak")
        print("4. Display")
        print("5. Exit")

        opt = input("Enter your option (1-5): ").strip()

        if opt == '1':
            val = input("Enter value to enqueue: ")
            q.nQ(val)
        elif opt == '2':
            q.dQ()
        elif opt == '3':
            front_val = q.peak()
            if front_val is not None:
                print(f"Peak element (Front): {front_val}")
        elif opt == '4':
            q.display()
        elif opt == '5':
            print("Exiting program.")
            break
        else:
            print("Invalid option! Please choose a number between 1 and 5.")
```

```
            front_val = q.peak()
            if front_val is not None:
                print(f"Peak element (Front): {front_val}")
        elif opt == '4':
            q.display()
        elif opt == '5':
            print("Exiting program.")
            break
        else:
            print("Invalid option! Please choose a number between 1 and 5.")

if __name__ == "__main__":
    main()
```

Output:

```
Select Circular Queue Implementation:
1. Array (Fixed-size Python List)
2. Linked List
Enter choice (1 or 2): 2

--- Circular Queue Menu (Linked List Implementation) ---
1. nQ (Enqueue)
2. dQ (Dequeue)
3. Peak
4. Display
5. Exit
Enter your option (1-5): 3
Queue is empty.

--- Circular Queue Menu (Linked List Implementation) ---
1. nQ (Enqueue)
2. dQ (Dequeue)
3. Peak
4. Display
5. Exit
Enter your option (1-5): 5
Exiting program.
```

Explanation

- **nQ()** – Adds an element to the rear.
- **dQ()** – Removes an element from the front.
- **peak()** – Shows the front element.
- **display()** – Displays all elements.
- **is_empty()** – Checks whether the queue is empty.
- **is_full()** – Checks whether the array queue is full.
- In a circular queue, the last position is connected back to the first position.

Error location	Original code	The problem	**The Fix**

CircularQueueArray.__init__	`self.front = "-1"`
	`self.rear = "-1"`
		String values `"-1"` cause a `TypeError` when you try to perform math operations like `%` or `+ 1`.	Changed to integers:
			`self.front = -1`
			`self.rear = -1`

CircularQueueArray.nQ	if self.front == "-1:"	Typo containing a misplaced colon `:` inside the string quotation marks, rendering the comparison invalid.	Corrected the check to match the new integer state: `if self.front == -1:`

CircularQueueArray.dQ	`self.front = "-1"`
	`self.rear = "-1"`
		Resetting variables back to strings after the queue became empty broke future math operations.	Updated to reset back to integers:
			`self.front = -1`
			`self.rear = -1`

WEEK-7

AIM (A):Write a Python program to implement a Queue and its operations such as Enqueue, Dequeue, Peek, and Display using a menu-driven approach. Implement the Queue using both an Array and a Linked List data structure.

Input:

```
MAX = 5
queue = []
front = 0
rear = -1
def enqueue():
    global rear
    if rear == MAX - 1:
        print("Queue Overflow")
    else:
        value = int(input("Enter element: "))
        queue.append(value)
        rear += 1
        print("Element inserted successfully")
def dequeue():
    global front
    if front > rear:
        print("Queue Underflow")
    else:
        print("Deleted element:", queue[front])
        front +=1
def peek():
    if front > rear:
        print("Queue is empty")
    else:
        print("Front element:", queue[front])
def display():
    if front > rear:
        print("Queue is empty")
    else:
        print("Queue elements:")
        for i in range(front, rear + 1):
            print(queue[i], end=" ")
        print()
```

```
while True:
    print("\n--- QUEUE USING ARRAY ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        enqueue()
    elif choice == 2:
        dequeue()
    elif choice == 3:
        peek()
    elif choice == 4:
        display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

Output:

```
--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 1
Element inserted successfully

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 2
Element inserted successfully

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 1

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 3
Element inserted successfully

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 2

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements:
2 3

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated
```

Explanation:

This program implements a **FIFO (First In First Out) queue using an array/list** in Python with a menu-driven approach. The **enqueue operation** inserts a new element at the rear of the queue, while the **dequeue operation** removes an element from the front. The **peek operation** displays the front element without removing it from the queue. The **display operation** prints all the elements currently present from front to rear. The program also checks for **queue overflow and underflow** conditions before performing operations. The menu repeatedly accepts the user's choice and performs the selected operation until the user chooses Exit.

Errors Faced

While implementing the array queue, errors such as **Queue Overflow** can occur when the rear reaches the maximum queue size, and **Queue Underflow** can occur when trying to dequeue from an empty queue. Incorrect handling of the front and rear variables can also cause indexing problems or incorrect queue output. Another possible error is **ValueError**, which occurs when the user enters a non-numeric value where an integer is expected. Invalid menu choices can also cause incorrect operation selection, so an appropriate validation message is displayed.

LINKED LIST:

Input:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
front = None
rear = None
def enqueue():
    global front, rear
    value = int(input("Enter element: "))
    new_node = Node(value)
    if rear is None:
        front = rear = new_node
    else:
        rear.next = new_node
        rear = new_node
    print("Element inserted successfully")
def dequeue():
    global front, rear
    if front is None:
        print("Queue Underflow")
    else:
        print("Deleted element:", front.data)
        front = front.next
        if front is None:
            rear = None
def peek():
    if front is None:
        print("Queue is empty")
    else:
        print("Front element:", front.data)
def display():
    if front is None:
        print("Queue is empty")
    else:
        temp = front
        print("Queue elements:")
        while temp is not None:
            print(temp.data, end=" ")
            temp = temp.next
        print()
while True:
    print("\n--- QUEUE USING LINKED LIST ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        enqueue()
    elif choice == 2:
        dequeue()
    elif choice == 3:
        peek()
    elif choice == 4:
        display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

Output:

```
--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 2
Element inserted successfully

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 2
Element inserted successfully

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 2
```

```
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 4
Element inserted successfully

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 2

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements:
2 4

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated
```

Explanation

This program implements a **FIFO queue using a linked list** where every element is stored inside a node. Each node contains two parts: **data** and a reference to the next node. The front pointer points to the first node and the rear pointer points to the last node. During **enqueue**, a new node is created and attached at the rear, while during **dequeue**, the front node is removed and the front pointer is moved to the next node. The **peek** operation displays the data of the front node without deleting it, and **display** traverses the linked list from front to rear. Since memory is allocated dynamically, the linked-list queue does not require a fixed queue size.

Errors Faced

While implementing the linked-list queue, the main errors faced were related to **front and rear pointer management**. Queue Underflow can occur when dequeue or peek is performed while the queue is empty. Special care is required when deleting the last element because both front and rear must be changed to None. Incorrectly accessing a node or its next reference may result in an **AttributeError**. A **ValueError** may also occur if non-integer data is entered when the program expects an integer. Incorrect pointer updates can also cause elements to be skipped or the linked list to display incorrectly.

B .Write a Python program to implement a Circular Queue and its operations such as Enqueue, Dequeue, Peek, and Display using a menu-driven approach.

Input:

```
MAX = 5
queue = [None] * MAX
front = -1
rear = -1
def enqueue():
    global front, rear
    if (rear + 1) % MAX == front:
        print("Circular Queue Overflow")
        return
    value = int(input("Enter element: "))
    if front == -1:
        front = 0
        rear = 0
    else:
        rear = (rear + 1) % MAX
    queue[rear] = value
    print("Element inserted successfully")
def dequeue():
    global front, rear
    if front == -1:
        print("Circular Queue Underflow")
        return
    print("Deleted element:", queue[front])
    queue[front] = None
    if front == rear:
        front = -1
        rear = -1
    else:
        front = (front + 1) % MAX
def peek():
    if front == -1:
        print("Queue is empty")
    else:
        print("Front element:", queue[front])
def display():
    if front == -1:
        print("Queue is empty")
        return
    print("Circular Queue elements:")
    i = front
    while True:
        print(queue[i], end=" ")
        if i == rear:
            break
        i = (i + 1) % MAX
    print()
while True:
    print("\n--- CIRCULAR QUEUE ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        enqueue()
    elif choice == 2:
        dequeue()
    elif choice == 3:
        peek()
    elif choice == 4:
        display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

Output:

```
--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 3
Element inserted successfully

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 4
Element inserted successfully

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 5
Element inserted successfully
```

```
--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 3

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 4

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Circular Queue elements:
4 5

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated
```

Explanation:

This program implements a **circular queue using an array** and follows the FIFO principle. Unlike a normal linear queue, a circular queue allows the rear to move back to the beginning of the array when it reaches the last position. The **enqueue operation** inserts elements at the rear using the modulo operation, while **dequeue** removes elements from the front and moves the front circularly. The **peek operation** displays the current front element without deleting it, and **display** traverses the queue from front to rear while handling the circular arrangement. The condition (rear + 1) % MAX == front is used to identify a full circular queue, while front == -1 represents an empty queue.

Errors Faced :

While implementing the circular queue, errors can occur if the **full and empty conditions** are not handled correctly. Circular Queue Overflow occurs when the next position of the rear is equal to the front, while Underflow occurs when the queue is empty. Incorrect use of the modulo operator can cause the front or rear to move to the wrong position. The display operation may also produce an infinite loop if its stopping condition is incorrect, especially when the rear wraps around to the beginning of the array. A **ValueError** can occur when the user enters invalid non-numeric input, and invalid menu choices must also be handled properly

WEEK-8

1.Write a Python program to implement a Binary Tree with its operations using an Array and a Linked List.

ARRAY:

Input:

```
print(f"Index [i]: {self.tree[i]}")
```

Output:

```
Level-order: A B C D E F
```

Explanation:
The provided Python script implements a **Binary Tree using a sequential array (list)** rather than traditional interconnected node objects. It uses static mathematical formulas where any parent node residing at index i maps its left child to 2 * i + 1 and its right child to 2 * i + 2. The tree is instantiated with a fixed capacity, filling empty slots with None to maintain structural spacing. The methods set_root, set_left, and set_right insert characters by calculating these precise index boundaries. Finally, it uses recursive depth-first methods (preorder, inorder, postorder) to navigate branches, alongside a sequential array loop to print elements level by level.
Errors faced:
While this code runs successfully for dense test cases, it contains **se**vere structural and logical vulnerabilities for sparse trees. First, the level_order method falsely assumes a simple linear array scan matches real depth levels, which breaks entirely if the tree contains empty intermediate paths. Second, the code lacks parent-validation checks, meaning you can dangerously insert children under a parent index that is currently None. This disconnects nodes, causing the recursive traversals to completely skip these orphan elements. Lastly, the fixed-size allocation triggers strict Index out of range! errors instead of dynamically resizing the array when inserting deeper tree levels

B)LINKED LIST
Input:

```
while queue:
    node = queue.pop(0)
```

Output:

Explanation:
This Python code implements a standard **linked Binary Tree using distinct node objects** rather than a flat array structure. The Node class encapsulates a data value alongside left and right pointers that reference child node instances. The BinaryTree class acts as the structural manager, featuring a create_tree method that manually connects nodes to assemble a specific hierarchical tree shape. For traversal, the code utilizes recursive depth-first searches (preorder, inorder, postorder) that follow child pointers down until hitting a None base case. Finally, it implements a true Breadth-First Search via the level_order method, which utilizes a queue data structure to visit nodes layer by layer from top to bottom.
Errors faced:
This optimized implementation successfully resolves the performance bottlenecks**,** encapsulation leaks, and rigid hardcoding found in the previous code. By replacing the standard list with collections.deque and utilizing popleft(), the level-order operations drop from an expensive \(\mathcal{O}(N)\) memory shift to an efficient(mathcal{O}(1)) time complexity. Furthermore, wrapping the recursive traversals inside internal helper structures fixes the encapsulation flaw, enabling users to print the tree cleanly via tree.inorder() without manually passing the root reference. Finally, the introduction of a dynamic insert() loop replaces fragile manual link assignments, granting the tree the flexibility to build itself structurally and dynamically from any runtime dataset.

Week-9

AIM :Write a python programme to implement a binary search tree with its operations and traversals.
INPUT:

```
        return root
```

```
        return root
```

```
        self._postorder_recursive(root.right, res)
```

```
print(f"In-order after deleting 50: {bst.inorder()}")
```

OUTPUT:

Explanation :

- **Insert:** Adds a new value while maintaining the BST property. Smaller values go to the left; larger values go to the right.
- **Search:** Searches for a value and reports whether it exists.
- **Delete:** Removes a node, handling nodes with no children, one child, or two children.
- **Inorder traversal:** Visits Left → Root → Right. It displays values in ascending order.
- **Preorder traversal:** Visits Root → Left → Right.
- **Postorder traversal:** Visits Left → Right → Root.

Error Table:

S.No.	Error	Cause	Solution
1	`ValueError`	Invalid integer input	Enter a valid integer
2	Invalid choice	Choice outside 1–7	Select a valid menu option
3	Value not found	Value does not exist in BST	Enter an existing value
4	Duplicate value	Value already exists	Enter a unique value